SQL Case Statements

- Helps with creating flags in the data
- Helps with building logical conditions
- A case statement is normally created within the SELECT statements
- Helps to create a new / extra column.

Syntax:

```
SELECT
    CASE WHEN condition 1 THEN result 1    -> Only get result if condition is satisfied.
         WHEN condition 2 THEN result 2
    ELSE default - result    -> If no condition is satisfied.
    END AS new - column_name
FROM database.schema.table;
```

- You get the default - result when all the outlined conditions are not satisfied.

Sales

sale-ID	Store	Price Amount
1	Pep	100
2	Pep	50
3	MrPrice	80
4	Mrprice	20

Flag:
0-40 low price
41-60 med price
> 60 High price

```
SELECT sale_ID, Store, Price,
    CASE
        WHEN price Between 0 AND 40 THEN 'Low price'
        WHEN price Between 41 AND 60 THEN 'med price'
    ELSE 'High Price'
    END AS Price-group
FROM Sales;
```

sale_ID	Store	Price	Price Group
1	Pep	100	High price
2	Pep	50	med price
3	MrPrice	80	High price
4	MrPrice	20	Low price

① SELECT *
CASE
WHEN Price > 1000 THEN 'Expensive'
WHEN Price BETWEEN 100 AND 1000 THEN 'mid-range'
Else 'Budget'
END AS product_category;
FROM Products;

Product-ID	Product-name	Price	Product-category
1	Laptop	1200.00	Expensive
2	Phone	800.00	mid-range
3	Keyboard	45.00	Budget
4	Monitor	300.00	mid-range
5	Mouse	25.00	Budget

e.g.
② SELECT customer-name,
Amount,
CASE
WHEN amount > 1000 THEN 'High-value'
WHEN amount between 500 AND 999.99 THEN 'mid-value'
ELSE 'Low-value'
END as order-value-category
FROM orders;

ORDERS

Customer ID	Customer-name	Amount
1	Alice	150.00
2	Bob	560.00
3	Charlie	999.99
4	Diana	45.50
5	Ethan	1200.00

customer-name	Amount	ordervalue-cat
Alice	150.00	Low value
Bob	560.00	medium value
Charlie	999.99	Medium value
Diana	45.50	Low value
Ethan	1200.00	High value

CASE Statement

3. Employees

emp-ID	emp-name	department	Salary	Position-level
1	John	IT	85 000	Senior IT
2	Sara	HR	60 000	Experienced HR
3	Mark	IT	75000	Staff
4	Lucy	finance	95000	Staff
5	Tom	HR	55000	Staff

```
SELECT emp_name,
       department,
       salary,
CASE
    WHEN department = 'IT' AND salary >80000
         THEN 'Senior IT'
    WHEN department = 'HR' AND salary >55000
         THEN 'Experienced HR'
    ELSE 'Staff'
  END AS position-level
FROM employees;
```

4. Table: `students`

student_id	student_name	score
1	Anna	92
2	Ben	76
3	Cara	59
4	David	83
5	Ella	68

Question:

Assign a letter grade:

- ≥ 90: 'A'
- 80–89: 'B'
- 70–79: 'C'
- 60–69: 'D'
- < 60: 'F'

Student_name	Score	grade
Anna	92	A
Ben	76	c
Cara	59	f
David	83	B
Ella	68	D

Expected Output Columns:

- `student_name`
- `score`
- `grade`

```
SELECT student_name,
       score,
CASE
    WHEN score ≥ 90 THEN 'A'
    WHEN score BETWEEN 80 AND 89 THEN 'B'
    WHEN score between 70 AND 79 THEN 'C'
    WHEN score between 60 AND 69 THEN 'D'
    WHEN score < 60 THEN 'f'
  END AS 'grade'
FROM students;
```

5. Table: `deliveries`

delivery_id	delivery_time_minutes
1	45
2	80
3	30
4	65
5	100

Question:

Label delivery performance:

- ≤ 30 mins: 'Fast'
- 31–60 mins: 'On Time'
- 60 mins: 'Late'

Expected Output Columns:

- `delivery_id`
- `delivery_time_minutes`
- `performance`

delivery-id	delivery-time-minutes	perfomance
1	45	On Time
2	80	late
3	30	fast
4	65	late
5	100	late

```
SELECT *
CASE
    WHEN delivery-time-minutes ≤ 30 mins
        THEN 'fAST'
    WHEN delivery-time-minutes between 31 AND
        60 minutes THEN 'ontime'
    ELSE 'Late'
END AS 'performance'
FROM deliveries;
```

6. Table: tickets

ticket_id	issue_type	priority
1	Login issue	1
2	Server down	3
3	Slow system	2
4	Email error	2
5	Password reset	1

Question:

Convert priority to labels:

- 3 → 'High'
- 2 → 'Medium'
- 1 → 'Low'

Expected Output Columns:

- `issue_type`
- `priority`
- `priority_label`

issue-type	Priority	Priority-level
Login issue	1	Low
Server down	3	High
Slow system	2	medium
Email error	2	Medium
Password reset	1	Low

```
SELECT issue-type,
       Priority,

CASE
    WHEN priority = 3 THEN 'High
    WHEN Priority = 2 THEN 'Medium'
    WHEN Priority = 1 THEN 'Low'
END AS 'Priority-label'
FROM tickets;
```

7. Table: attendance

student_id	days_present	total_days
1	45	50
2	30	50
3	48	50
4	25	50
5	50	50

Question:

Calculate attendance % and classify:

- ≥ 90% → 'Excellent'
- 75–89% → 'Good'
- < 75% → 'Needs Improvement'

Expected Output Columns:

- `student_id`
- `attendance_percentage`
- `attendance_status`

Student-id	att-%	attendance-status
1	90	Excellent
2	60	Needs Improvement
3	96	Excellent
4	50	Needs Improvement
5	100	Excellent

```
SELECT student-id,
       attendance-percentage,
CASE
    WHEN (days-present * 100 /total-days ) >= 90
         THEN 'Excellent'
    WHEN (days-present * 100 / total-days) BETWEEN 75
      AND 89      THEN 'Good'
    ELSE 'Needs Improvement'
END AS attendance-status
FROM attendance;
```

8. Table: `products_inventory`

product_id	stock_qty
1	5
2	0
3	25
4	10
5	3

Question:

Label stock status:

- 0 → 'Out of Stock'
- 1–5 → 'Low Stock'
- 5 → 'In Stock'

Expected Output Columns:

- `product_id`
- `stock_qty`
- `stock_status`

Product_id	stock_qty	Stock_status
1	5	Low stock
2	0	Out of stock
3	25	In stock
4	10	In Stock
5	3	Low stock

```
SELECT *
CASE
  WHEN stock_qty = 0 THEN 'Out of Stock'
  WHEN stock_qty between 1 AND 5 THEN 'Low Stock'
  ELSE 'In stock'
END AS 'Stock_status'
FROM products_inventory;
```

9. Table: `classes`

class_id	~~subject~~	enrolled_students
1	Math	30
2	English	25
3	Science	15
4	Art	5
5	History	20

Question:

Classify by size:

- ≥ 25 → 'Large'
- 10–24 → 'Medium'
- < 10 → 'Small'

Expected Output Columns:

- `subject`
- `enrolled_students`
- `class_size_category`

Subject	enrolled-students	class-size-category
Maths	30	Large
English	25	Large
Science	15	Medium
Art	5	Small
History	20	Medium

```
SELECT subjects
       enrolled-students
CASE
    WHEN enrolled-students ≥ 25   THEN   'Large'
    WHEN enrolled-students between  10  AND  24
             THEN  'Medium'
    Else  'Small'
END AS 'class-size-category'
FROM classes;
```

SQL query to find emplo

10. Table: payments

payment_id	amount	payment_method
1	50.00	Card
2	200.00	Cash
3	150.00	Card
4	75.00	PayPal
5	300.00	Cash

Question:

Apply discount flag:

- If `payment_method = 'Cash'` and amount ≥ 200 → 'Eligible for Discount'
- Otherwise → 'Not Eligible'

Expected Output Columns:

- `payment_id`
- `payment_method`
- `amount`
- `discount_eligibility`

```
SELECT *
CASE
    WHEN payment-method ='Cash' AND amount ≥ 200
        THEN 'Eligible for Discount'
    ELSE 'Not Eligible'
END AS 'discount-eligibility'

FROM payments;
```

Payment-id	Payment-Method	amount	discount-Eligibility
1	card	50.00	Not Eligible
2	cash	200.00	Eligible for Discount
3	card	150.00	Not Eligible
4	Pay-pal	75.00	Not Eligible
5	cash	300.00	Eligible for Discount